CS216: Bitcoin Transaction Lab Assignment

Legacy vs SegWit Transaction Analysis

Team: Break The Chain

Boddu Kunal	240003020
Shaik Riyaj	240001068
Sangati Chakradhar Reddy	240001063
Kesavarapu Deepak Reddy	240041022

Department of Computer Science and Engineering

Indian Institute of Technology Indore

March 10, 2026

Abstract

Bitcoin is a decentralized digital currency that enables peer-to-peer transactions without the need for a central authority. Each transaction in the Bitcoin network must be validated through a cryptographic scripting system to ensure that only the rightful owner of funds can spend them.

Over time the Bitcoin protocol has evolved to improve scalability, security, and efficiency. One of the most significant upgrades introduced to the network is Segregated Witness (SegWit), which modifies the transaction structure by separating signature data from the main transaction body.

In this assignment we implemented both Legacy transactions (Pay-to-Public-Key-Hash or P2PKH) and SegWit transactions (Pay-to-Script-Hash wrapped Pay-to-Witness-Public-Key-Hash or P2SH-P2WPKH) using a local Bitcoin regtest network.

Python programs were written to interact with the Bitcoin daemon (`bitcoind`) through Remote Procedure Calls (RPC). Using these programs, transaction chains were created between three addresses (A $\rightarrow$ B $\rightarrow$ C) for both transaction types.

The generated transactions were decoded to examine the locking and unlocking scripts involved in transaction validation. Additionally, the Bitcoin Script debugger (`btcdeb`) was used to step through script execution and analyze how stack operations validate the transaction.

Finally, we compared the transaction sizes, virtual sizes, and weight metrics of Legacy and SegWit transactions to analyze the efficiency benefits introduced by SegWit.

1 Introduction

Bitcoin is a decentralized peer-to-peer payment system that allows users to transfer value without relying on banks or financial institutions. Instead of trusting a central authority, Bitcoin relies on a distributed network of nodes that collectively verify and record transactions in a public ledger known as the blockchain.

Each Bitcoin transaction consumes previously created outputs and generates new outputs. These outputs can only be spent if the spender satisfies the conditions specified in the output's locking script. This verification process is performed using a simple stack-based programming language known as Bitcoin Script.

The earliest and most widely used transaction type in Bitcoin is Pay-to-Public-Key-Hash (P2PKH). In this format, the locking script contains the hash of the recipient's public key. To spend the output, the user must provide a valid digital signature and the corresponding public key.

Although P2PKH transactions are secure and widely supported, they include signature data directly inside the transaction input. As Bitcoin adoption increased, this design contributed to larger transaction sizes and increased transaction fees.

To address these issues, the Bitcoin protocol introduced Segregated Witness (SegWit) in 2017 through Bitcoin Improvement Proposal (BIP 141). SegWit modifies the transaction format by separating signature data into a dedicated witness structure. This reduces the effective size of transactions and improves network scalability.

The goal of this assignment is to understand the structural and functional differences between Legacy and SegWit transactions.

2 Objectives of the Assignment

The primary objective of this assignment is to gain hands-on experience with Bitcoin transaction creation and validation. Rather than relying solely on theoretical explanations, this experiment focuses on interacting directly with a Bitcoin node and constructing transactions programmatically.

The key objectives of this assignment are as follows:

- To create Legacy (P2PKH) transactions using RPC commands and analyze their locking and unlocking scripts.
- To generate SegWit (P2SH-P2WPKH) transactions and observe how signature data is separated into the witness structure.
- To decode raw transactions and examine the script structures used for transaction validation.
- To use the Bitcoin Script debugger (`btcdeb`) to step through script execution and understand how stack operations validate transaction conditions.
- To compare the size, weight, and virtual size of Legacy and SegWit transactions and analyze the efficiency improvements introduced by SegWit.

Through this practical experiment we aim to develop a deeper understanding of Bitcoin's scripting system, transaction validation mechanism, and the importance of protocol upgrades such as SegWit for improving the scalability and efficiency of the Bitcoin network.

3 Core Concepts

This experiment involves the creation and validation of Bitcoin transactions using Legacy (P2PKH) and SegWit (P2SH-P2WPKH) formats. To understand the transaction workflow, it is necessary to briefly review the UTXO model, Bitcoin Script, and the SegWit upgrade.

3.1 The UTXO Model

Bitcoin uses the Unspent Transaction Output (UTXO) model instead of maintaining account balances. Each transaction consumes previously created outputs and generates new outputs that can later be spent.

Transaction inputs reference earlier outputs, while transaction outputs define the conditions under which the coins can be spent. A transaction is valid only if the referenced outputs exist, remain unspent, and the unlocking script satisfies the locking conditions defined by the previous transaction.

3.2 Bitcoin Script

Bitcoin Script is a stack-based scripting language used to validate transactions. Each transaction contains two scripts: the locking script (`scriptPubKey`) and the unlocking script (`scriptSig`). During validation, the unlocking script is executed first, followed by the locking script. If the final result of the script execution evaluates to TRUE, the transaction is considered valid.

3.3 Pay-to-Public-Key-Hash (P2PKH)

In a P2PKH transaction, the locking script contains the hash of the recipient's public key. To spend the output, the spender must provide a valid digital signature and the corresponding public key.

The typical locking script is:

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

The unlocking script provides the signature and public key required to verify ownership of the funds.

3.4 Segregated Witness (SegWit)

Segregated Witness (SegWit) modifies the structure of Bitcoin transactions by separating signature data from the main transaction body. Instead of storing signatures inside the unlocking script, they are moved into a separate witness field.

This reduces the effective transaction size, lowers fees, and improves block capacity. In this assignment we use the P2SH-P2WPKH format, which embeds SegWit functionality inside a P2SH address.

3.5 Virtual Size and Weight

SegWit introduces a new metric called transaction weight:

$$Weight = (BaseSize \times 4) + WitnessSize$$

The effective size used for fee calculation is the virtual size:

$$vsize = \frac{weight}{4}$$

Because witness data is discounted, SegWit transactions usually have a smaller virtual size than Legacy transactions, making them more efficient in terms of block space and transaction fees.

4 Legacy Transaction Implementation (P2PKH)

The first part of the experiment focuses on creating and analyzing Bitcoin transactions using the Legacy Pay-to-Public-Key-Hash (P2PKH) format on a local Bitcoin regtest network. Three Legacy addresses were generated and labeled as Address A, Address B, and Address C. Address A was first funded using coins mined in regtest mode. Two transactions were then created to form the chain:

$$A \rightarrow B \rightarrow C$$

4.1 Transaction A → B

```
[======== TRANSACTION A → B ========
 Fee   : 0.00000225 BTC
 Size  : 225 bytes
 vsize : 225 vbytes
 Weight: 900

 [Locking Scripts - scriptPubKey]
   Output 0 (n1Bzh84DBVcFAB7RPt3NJHR45pJ7MTqZXP):
     Type: pubkeyhash
     ASM : OP_DUP OP_HASH160 d7cc9d2a552ecb5909f3fe56540dd1921446713d OP_EQUALVERIFY OP_CHECKSIG
     HEX : 76a914d7cc9d2a552ecb5909f3fe56540dd1921446713d88ac
   Output 1 (n48X9VFzPYQobugsqL1KytUv3wWQqLTZUZ):
     Type: pubkeyhash
     ASM : OP_DUP OP_HASH160 f80cbfe7fb3790b52e0bb43a6a25c483740a7de0 OP_EQUALVERIFY OP_CHECKSIG
     HEX : 76a914f80cbfe7fb3790b52e0bb43a6a25c483740a7de088ac

 TXID A→B: d0e8703a6b3dc07af082cdf6661130458999e8fb51d10e08c3652715edbd546c
```

Figure 1: Decoded Legacy transaction from Address A to Address B

Figure 1 shows the decoded raw transaction for the transfer from Address A to Address B. The output contains the locking script (`scriptPubKey`) that defines the conditions required to spend the output. In a P2PKH transaction the locking script follows the standard format:

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

This script specifies that the spender must provide a valid digital signature and a public key whose hash matches the stored public key hash in order to unlock the output.

4.2 Transaction B → C

```
========= TRANSACTION B → C =========
Fee   : 0.00000225 BTC
Size  : 225 bytes
vsize : 225 vbytes
Weight: 900

[Unlocking Scripts - scriptSig]
  ASM : 304402205556af178ee4527188828 5d9d2345c6289996e5a98482125c9f60f6a8f7f83c50220077c357128313be4ce606cf0a2304b9b96f356dfdca4f4119491572e05ec3622[ALL] 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
  HEX : 473044022055 56af178ee4527188828 5d9d2345c6289996e5a98482125c9f60f6a8f7f83c50220077c357128313be4ce606cf0a2304b9b96f356dfdca4f4119491572e05ec362201210353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc

TXID B→C: d2eb56eb78611a1bf90d113be95b8e3cb734e618d830a1546b5d29ec3570c110
```

Figure 2: Decoded Legacy transaction from Address B to Address C

Figure 2 shows the decoded transaction that spends the output generated in the previous transaction. The UTXO belonging to Address B is used as the input for the next transaction, which transfers coins to Address C. The unlocking script (`scriptSig`) contains the digital signature and the public key required to satisfy the locking conditions defined in the previous transaction.

4.3 Script Execution using btcdeb

```
(base) kunalb@Kunals-MacBook-Air CS216-BreakTheChain-Legacy-Segwit % btcdeb '[304402205556af178ee4527188828 5d9d2345c6289996e5a98482125c9f60f6a8f7f83c50220077c357128313be4ce606cf0a2304b9b96f356dfdca4f4119491572e05ec3622 0353fb65349f204660048c
32717554565e195b3793367d5dcee4599e5d97b176cc OP_DUP OP_HASH160 d7ce9d2a552ecb5909f3fe56540dd1921446713d OP_EQUALVERIFY OP_CHECKSIG]'
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
7 op script loaded. type `help` for usage information
script                                                              |  stack
--------------------------------------------------------------------+--------
304402205556af178ee4527188828 5d9d2345c6289996e5a98482125c9f60f6... |
0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc |
OP_DUP                                                              |
OP_HASH160                                                          |
d7ce9d2a552ecb5909f3fe56540dd1921446713d                            |
OP_EQUALVERIFY                                                      |
OP_CHECKSIG                                                         |
#0000 304402205556af178ee4527188828 5d9d2345c6289996e5a98482125c9f60f6a8f7f83c50220077c357128313be4ce606cf0a2304b9b96f356dfdca4f4119491572e05ec3622
```

Figure 3: Initial script loaded in btcdeb

Figure 3 shows the initial script loaded in the Bitcoin Script debugger (`btcdeb`). The unlocking script and locking script are combined and loaded into the execution environment. At this stage the script instructions are visible but have not yet been executed. The stack is initially empty and will be populated during execution.

```
btcdeb> step
                <> PUSH stack 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6a8f7f83c50220077c357128313be4ce606cf0a2304b9b96f356dfdca4f4119491572e05ec3622
script                                                                  |                                                                stack
------------------------------------------------------------------------+----------------------------------------------------------------
0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc | 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6...
OP_DUP                                                                  |
OP_HASH160                                                              |
d7cc9d2a552ecb5909f3fe56540dd1921446713d                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0001 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
btcdeb> step
                <> PUSH stack 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
script                                                                  |                                                                stack
------------------------------------------------------------------------+----------------------------------------------------------------
OP_DUP                                                                  | 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
OP_HASH160                                                              | 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6...
d7cc9d2a552ecb5909f3fe56540dd1921446713d                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0002 OP_DUP
btcdeb> step
                <> PUSH stack 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
script                                                                  |                                                                stack
------------------------------------------------------------------------+----------------------------------------------------------------
OP_HASH160                                                              | 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
d7cc9d2a552ecb5909f3fe56540dd1921446713d                                | 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
OP_EQUALVERIFY                                                          | 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6...
OP_CHECKSIG                                                             |
#0003 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack d7cc9d2a552ecb5909f3fe56540dd1921446713d
script                                                                  |                                                                stack
------------------------------------------------------------------------+----------------------------------------------------------------
d7cc9d2a552ecb5909f3fe56540dd1921446713d                                |                                d7cc9d2a552ecb5909f3fe56540dd1921446713d
OP_EQUALVERIFY                                                          | 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
OP_CHECKSIG                                                             | 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6...
#0004 d7cc9d2a552ecb5909f3fe56540dd1921446713d
btcdeb> step
                <> PUSH stack d7cc9d2a552ecb5909f3fe56540dd1921446713d
script                                                                  |                                                                stack
------------------------------------------------------------------------+----------------------------------------------------------------
OP_EQUALVERIFY                                                          |                                d7cc9d2a552ecb5909f3fe56540dd1921446713d
OP_CHECKSIG                                                             |                                d7cc9d2a552ecb5909f3fe56540dd1921446713d
                                                                        | 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
                                                                        | 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6...
#0005 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                                  |                                                                stack
------------------------------------------------------------------------+----------------------------------------------------------------
OP_CHECKSIG                                                             | 0353fb65349f204660048c32717554565e195b3793367d5dcee4599e5d97b176cc
                                                                        | 304402205556af178ee45271888285d9d2345c6289996e5a98482125c9f60f6...
#0006 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
error: Signature is found in scriptCode
```

Figure 4: Step-by-step execution of the Legacy transaction script

Figure 4 shows the step-by-step execution of the Bitcoin Script using btcdeb. During execution, the unlocking script first pushes the signature and public key onto the stack. The locking script then performs a sequence of operations to verify ownership of the funds.

The instructions execute as follows:

- `OP_DUP` duplicates the public key on the stack
- `OP_HASH160` computes the HASH160 of the public key
- `OP_EQUALVERIFY` checks whether the computed hash matches the expected public key hash
- `OP_CHECKSIG` verifies the digital signature using the provided public key

If the computed hash matches and the signature verification succeeds, the script returns TRUE and the transaction is considered valid.

In the debugging environment, `OP_CHECKSIG` may produce an error because btcdeb does not always have access to the full transaction context required for signature verification. However, the preceding steps confirm that the unlocking data correctly matches the locking conditions.

5 P2SH-SegWit Transaction Implementation

In the second part of the experiment, transactions were created using the SegWit address format (P2SH-P2WPKH). The objective was to observe how SegWit modifies the structure of Bitcoin transactions and how signature data is stored separately from the main transaction body.

Similar to the Legacy experiment, three SegWit addresses were generated and labeled as Address A', Address B', and Address C'. Address A' was funded using coins mined on the regtest network. Two transactions were then created to form the chain:

$$A' \rightarrow B' \rightarrow C'$$

5.1 Transaction A' → B'

```
========= TRANSACTION A → B =========
Fee   : 0.00000166 BTC
Size  : 247 bytes
vsize : 166 vbytes
Weight: 661

[Locking Scripts - scriptPubKey]
  Output 0 (2MznZeErXs4xoMMM8BqkDTv3fyDs644zbee):
    Type: scripthash
    ASM : OP_HASH160 52b62e9600ee22e6a9ad0b2ba782b77759756e95 OP_EQUAL
    HEX : a91452b62e9600ee22e6a9ad0b2ba782b77759756e9587
  Output 1 (2MsuvbnDs6sshJnujwbxb3tUBRrMgRvZtrq):
    Type: scripthash
    ASM : OP_HASH160 0751b52edbe7e46b0f5d9eb13801cc6670f26256 OP_EQUAL
    HEX : a9140751b52edbe7e46b0f5d9eb13801cc6670f2625687

TXID A→B: c010a7bd237c273b49a79634586b9c60f5be9ce140a73666d48333e14a72df73
```

Figure 5: Decoded SegWit transaction from Address A' to Address B'

Figure 5 shows the decoded raw transaction for the transfer from Address A' to Address B'. In a P2SH-P2WPKH transaction the locking script follows the Pay-to-Script-Hash format:

```
OP_HASH160 <ScriptHash> OP_EQUAL
```

Instead of directly containing the public key hash, the script stores the hash of a redeem script. The actual signature and public key required for validation are stored separately in the witness structure.

This separation of signature data is the key feature of SegWit and allows the effective transaction size to be reduced.

5.2 Transaction B' → C'

```
========= TRANSACTION B → C =========
Fee   : 0.00000166 BTC
Size  : 247 bytes
vsize : 166 vbytes
Weight: 661

[Unlocking Scripts — scriptSig + Witness]
  ScriptSig ASM : 00144668 0dcd44884176c88bea950b27293305386602
  ScriptSig HEX : 16001446680dcd44884176c88bea950b27293305386602
  Witness[0] (sig)    : 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef1074c1a8f13022041396dd140ca91901e269dad0721369da3680e193a82abb9310fd91fb8368e8001
  Witness[1] (pubkey) : 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511

TXID B→C: f251aa2bf6fe3c5713f9c1d0b34f62e9214bc349b7254bfb205821b0d630f035
```

Figure 6: Decoded SegWit transaction from Address B' to Address C'

Figure 6 shows the second SegWit transaction where the output created in the previous transaction is used as the input for the next transfer.

Unlike Legacy transactions, the unlocking data is not stored in the `scriptSig`. Instead, the signature and public key are placed inside the `txinwitness` field. This structure separates witness data from the main transaction body, which reduces the effective size of the transaction when calculating fees.

5.3 Script Execution using btcdeb

```
(base) kunalbdkunals-MacBook-Air ~$ cd ~/Breakthrough/chain-legacy-segwit % btcdeb '3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef1074c1a8f13022041396dd140ca91901e269dad0721369da3680e193a82abb9310fd91fb8368e8001 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511 OP_DUP OP_HASH160 46680dcd44884176c88bea950b27293305386602 OP_EQUALVERIFY OP_CHECKSIG'
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
7 op script loaded. type `help` for usage information
script                                                  |  stack
--------------------------------------------------------|--------
3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10... |
03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511 |
OP_DUP                                                  |
OP_HASH160                                              |
46680dcd44884176c88bea950b27293305386602                |
OP_EQUALVERIFY                                          |
OP_CHECKSIG                                             |
#0000 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef1074c1a8f13022041396dd140ca91901e269dad0721369da3680e193a82abb9310fd91fb8368e80
```

Figure 7: Initial SegWit script loaded in btcdeb

Figure 7 shows the script loaded in the btcdeb debugger before execution begins. The unlocking script and locking script are combined and prepared for step-by-step execution.

```
btcdeb> step
                <> PUSH stack 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef1074c1a8f13022041396dd140ca91901e269dad0721369da3680e193a82abb9310fd91fb8368e80
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511 | 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10...
OP_DUP                                                                  |
OP_HASH160                                                              |
46680dcd44884176c88bea950b27293305386602                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0001 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
btcdeb> step
                <> PUSH stack 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_DUP                                                                  | 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
OP_HASH160                                                              | 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10...
46680dcd44884176c88bea950b27293305386602                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0002 OP_DUP
btcdeb> step
                <> PUSH stack 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_HASH160                                                              | 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
46680dcd44884176c88bea950b27293305386602                                | 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
OP_EQUALVERIFY                                                          | 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10...
OP_CHECKSIG                                                             |
#0003 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 46680dcd44884176c88bea950b27293305386602
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
46680dcd44884176c88bea950b27293305386602                                |                                 46680dcd44884176c88bea950b27293305386602
OP_EQUALVERIFY                                                          | 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
OP_CHECKSIG                                                             | 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10...
#0004 46680dcd44884176c88bea950b27293305386602
btcdeb> step
                <> PUSH stack 46680dcd44884176c88bea950b27293305386602
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_EQUALVERIFY                                                          |                                 46680dcd44884176c88bea950b27293305386602
OP_CHECKSIG                                                             |                                 46680dcd44884176c88bea950b27293305386602
                                                                        | 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
                                                                        | 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10...
#0005 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_CHECKSIG                                                             | 03454eaea3b8ab8761fafc1a60316b94a49720339f47fe261294c8eabc8566a511
                                                                        | 3044022003aa4386d7ea35f59f83f68677723cd18fd78117c9b87f585c6ef10...
#0006 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
error: Signature is found in scriptCode
```

Figure 8: Step-by-step execution of the SegWit transaction script

Figure 8 shows the execution of the SegWit transaction script in the btcdeb debugger. Similar to Legacy transactions, the unlocking data first pushes the signature and public key onto the stack.

The script then executes the following operations:

- `OP_DUP` duplicates the public key
- `OP_HASH160` computes the HASH160 of the public key
- `OP_EQUALVERIFY` verifies that the computed hash matches the expected public key hash
- `OP_CHECKSIG` verifies the digital signature

If the hash comparison and signature verification succeed, the script returns TRUE and the transaction is considered valid.

Similar to the Legacy debugging process, btcdeb may display an error during the execution of `OP_CHECKSIG` because the debugger does not always have access to the complete transaction context required for signature verification. However, the previous operations confirm that the unlocking data matches the locking conditions defined in the script.

6 Comparative Analysis of Legacy and SegWit Transactions

After implementing both Legacy (P2PKH) and SegWit (P2SH-P2WPKH) transactions, a comparative analysis was performed to evaluate their structural differences and efficiency. The analysis focuses on three primary metrics: raw transaction size, virtual size (vsize), and transaction weight.

These metrics were obtained by decoding the raw transactions generated during the experiment and extracting the corresponding fields using the `decoderawtransaction` RPC command.

6.1 Transaction Size Comparison

Table 1 summarizes the size metrics obtained for the Legacy and SegWit transaction chains.

Transaction	Size (bytes)	vsize (vbytes)	Weight
Legacy A → B	225	225	900
Legacy B → C	225	225	900
SegWit A' → B'	247	166	661
SegWit B' → C'	247	166	661

Table 1: Transaction size comparison between Legacy and SegWit

From the table it can be observed that SegWit transactions have a slightly larger raw size compared to Legacy transactions. However, the virtual size (vsize) of SegWit transactions

is significantly smaller.

6.2 Average Transaction Size

To compare the efficiency of both transaction formats, the average virtual size was calculated.

$$Average_{Legacy} = \frac{225 + 225}{2} = 225 \text{ vbytes}$$

$$Average_{SegWit} = \frac{166 + 166}{2} = 166 \text{ vbytes}$$

The percentage reduction in transaction size can be calculated as:

$$Savings = \frac{225 - 166}{225} \times 100$$

$$Savings \approx 26.2\%$$

This result shows that SegWit transactions reduce the effective transaction size by approximately 26% compared to Legacy transactions.

6.3 Weight Calculation

SegWit introduces a new metric called transaction weight. The weight of a transaction is calculated using the formula:

$$Weight = (BaseSize \times 4) + WitnessSize$$

The effective size used for fee calculation is the virtual size:

$$vsize = \frac{Weight}{4}$$

Because witness data is counted at one quarter of the weight of normal transaction data,

SegWit transactions achieve a lower virtual size even if their raw byte size appears slightly larger.

6.4 Script Structure Comparison

The main structural difference between Legacy and SegWit transactions lies in how unlocking data is stored.

Feature	Legacy (P2PKH)	SegWit (P2SH-P2WPKH)
Locking Script	OP_DUP OP_HASH160 ...	OP_HASH160 ...
Unlocking Data	scriptSig	Witness Field
Signature Location	scriptSig	txinwitness
Transaction Malleability	Possible	Fixed
Fee Efficiency	Lower	Higher

Table 2: Structural comparison of Legacy and SegWit scripts

7 Benefits of SegWit

Segregated Witness was introduced to address several limitations of Legacy Bitcoin transactions. The experimental results obtained in this assignment demonstrate the practical benefits of the SegWit transaction format.

7.1 Lower Transaction Fees

Because SegWit transactions have a smaller virtual size, they require fewer vbytes to be included in a block. Since transaction fees are calculated based on virtual size, SegWit transactions typically require lower fees than equivalent Legacy transactions.

This improvement benefits users by reducing the cost of transacting on the Bitcoin network.

7.2 Increased Block Capacity

By discounting witness data during weight calculation, SegWit allows more transactions to fit within a single block. This effectively increases the throughput of the Bitcoin network without requiring an increase in the maximum block size.

As a result, the network can process a larger number of transactions per block, which improves scalability.

7.3 Fix for Transaction Malleability

One of the most important improvements introduced by SegWit is the elimination of transaction malleability. In Legacy transactions, certain parts of the transaction data could be modified without invalidating the signature, which could change the transaction ID.

SegWit resolves this issue by separating the signature data from the transaction hash calculation. As a result, transaction IDs become stable and cannot be altered through signature manipulation.

7.4 Support for Future Upgrades

SegWit also improves the flexibility of the Bitcoin protocol by making it easier to introduce future upgrades. Because witness data is stored separately, new script features can be introduced without affecting the existing transaction format.

This design allows the Bitcoin network to evolve while maintaining backward compatibility.

8 Discussion

The results obtained in this experiment demonstrate the practical differences between Legacy and SegWit transaction formats. Although SegWit transactions appear slightly larger in terms of raw byte size, their effective size when measured in virtual bytes is significantly

smaller.

This difference arises from the witness discount mechanism introduced by SegWit. By assigning a lower weight to signature data, SegWit reduces the effective size used for transaction fee calculation.

The script analysis performed using the btcdeb debugger also illustrates how both transaction formats rely on similar cryptographic verification mechanisms. In both cases the unlocking script provides the signature and public key, while the locking script verifies ownership using hash comparison and signature validation.

However, SegWit improves the transaction structure by moving the signature data to the witness field. This not only reduces the effective transaction size but also improves the overall design of the protocol.

The experimental workflow implemented in this assignment demonstrates how raw transactions can be created, decoded, and analyzed programmatically using RPC commands. It also highlights the usefulness of debugging tools such as btcdeb for understanding the execution of Bitcoin Script.

9 Conclusion

This assignment explored the process of creating and validating Bitcoin transactions using both Legacy (P2PKH) and SegWit (P2SH-P2WPKH) address formats.

Using Python scripts and RPC communication with a local Bitcoin regtest node, two transaction chains were created for each address format. The transactions were decoded to analyze their structure and scripts, and the execution of the scripts was examined using the btcdeb debugger.

The comparative analysis shows that although SegWit transactions may have a slightly larger raw byte size, their effective size measured in virtual bytes is significantly smaller. In the experiment conducted, SegWit transactions reduced the virtual size by approximately 26% compared to Legacy transactions.

This reduction is achieved through the witness discount mechanism, where signature data stored in the witness field is counted with a lower weight. As a result, SegWit transactions are more efficient in terms of block space usage and transaction fees.

In addition to improving efficiency, SegWit also fixes transaction malleability and enables future protocol upgrades. These improvements make SegWit a significant milestone in the evolution of the Bitcoin protocol.

Overall, this assignment provided practical insight into how Bitcoin transactions are constructed, validated, and optimized through modern transaction formats.